DEPARTEMEN TEKNIK INFORMATIKA — INSTITUT TEKNOLOGI SEPULUH NOPEMBER (ITS)

Odd Semester 2026/2027 · Open-Book Group Midterm Project

EF234301 WEB PROGRAMMING (D)
MIDTERM PROJECT REPORT

Project Topic 3: Freelance Job Board & Candidate Application Portal

Laravel 11.x MVC | PHP 8.3.33 | SQLite / MySQL | Three.js WebGL | Responsive Blade UI

Instructors: MM Irfan Subakti [yifana@gmail.com]
Teaching Assistants: Callista Fidelya Roba Gultom, Kevin Andreas

Group Project Members & Contributions:
• **Rianti Dewi** (5025231001) — **50%** (Database Migrations, Models, Auth, Features 1, 2, 3, 8)
• **Darminto Utomo** (5025231002) — **50%** (Features 4, 5, 6, 7, Responsive UI, Unit Tests, Deployment)

GitHub Repository: https://github.com/PopSharkkk/WebProgramming
Deployed Live Website: http://127.0.0.1:8000 (Local Host & Cloud Staging)

Date of Submission: 28 October 2026

TABLE OF CONTENTS

1. **Executive Summary & Domain Overview**
2. **Database Design & Architecture [Grading Rubric: 20 Points]**
    - 2.1 Conceptual Data Model (CDM)
    - 2.2 Physical Data Model (PDM) & Relational Schema
    - 2.3 Foreign Keys & Referential Integrity Constraints
3. **Implementation & MVC Architecture [Grading Rubric: 50 Points]**
    - 3.1 Laravel MVC Architecture Pattern
    - 3.2 Authentication Subsystem (Login, Register, Logout)
    - 3.3 Detailed Breakdown of the 8 Core Features
    - 3.4 Route Mapping Matrix
    - 3.5 Automated Test Verification (12 Tests, 47 Assertions)
4. **Analysis and Evaluation [Grading Rubric: 25 Points]**
    - 4.1 MVC Architectural Separation & Modularity
    - 4.2 Eloquent ORM vs. Raw SQL & Eager Loading Audit
    - 4.3 Database Normalization Audit (1NF, 2NF, 3NF)
    - 4.4 Security Audit (CSRF, SQL Injection, XSS, RBAC, File Uploads)
    - 4.5 Limitations & Future Work
5. **Conclusion [Grading Rubric: 5 Points]**
6. **Individual Contributions Statement**
7. **Academic Integrity Pledge**

1. EXECUTIVE SUMMARY & DOMAIN OVERVIEW

The modern academic ecosystem harbors substantial student talent across disciplines such as software engineering, UI/UX design, technical writing, and peer academic tutoring. Local businesses and student organizations frequently seek flexible, cost-effective talent for project-based sprints. However, conventional enterprise recruitment platforms (e.g., LinkedIn, Indeed) impose significant overhead, lack student-specific validation, and are poorly calibrated for micro-engagements.

Ascend Gigs is a purpose-built *Freelance Job Board & Candidate Application Portal* engineered using the Laravel 11 MVC PHP framework. It provides a transparent, dual-sided marketplace facilitating student job discovery, direct pitch submissions with PDF CV resume attachments, structured employer applicant review workflows, interactive bookmark watchlists, and comprehensive company profile customization.

2. DATABASE DESIGN [RUBRIC: 20 POINTS]

2.1 Conceptual Data Model (CDM)

The Conceptual Data Model represents the business domain entities, their attributes, and their cardinalities without exposing physical implementation constraints or primary/foreign key mechanics.

- **User**: Represents authenticated participants, categorizing actors into either *Student Freelancers* or *Employers / Organizations*.
- **Employer**: Represents the organization or club hiring student freelancers, possessing company branding, bio, official website, and social links. (1:1 with User).
- **Job Posting**: Represents gig vacancies published by Employers, specifying work mode (Remote, On-site, Hybrid), budget, application deadline, and deliverables. (1:N from Employer).
- **Job Application**: Represents a student candidate's pitch, containing pitch narrative, portfolio hyperlink, PDF CV file, and hiring stage status (Pending, Interviewed, Hired, Rejected). (N:1 with Job Posting, N:1 with User).
- **Bookmark**: Represents saved opportunities in a student's personal watchlist. (M:N junction between User and Job Posting).

2.2 Physical Data Model (PDM) & Relational Schema

The physical schema is implemented with rigorous data types, indexes, and referential constraints across 5 relational tables:

Table	Column Name	Data Type & Modifiers	Key / Constraint	Description
users	`id`	BIGINT UNSIGNED AUTO_INCREMENT	PRIMARY KEY	Unique user identity
	`name`	VARCHAR(255) NOT NULL	-	Full candidate/representative name
	`email`	VARCHAR(255) NOT NULL	UNIQUE	Login credential email
	`password`	VARCHAR(255) NOT NULL	-	Bcrypt hashed password string
	`role`	ENUM('freelancer', 'employer')	DEFAULT 'freelancer'	Role-based access control selector
	`phone`	VARCHAR(255) NULL	-	Optional contact telephone
employers	`id`	BIGINT UNSIGNED AUTO_INCREMENT	PRIMARY KEY	Unique employer identity
	`user_id`	BIGINT UNSIGNED NOT NULL	FOREIGN KEY → users(id)	Cascade delete owner link
	`company_name`	VARCHAR(255) NOT NULL	-	Official studio/club name

Table	Column Name	Data Type & Modifiers	Key / Constraint	Description
	`industry`	VARCHAR(255) NOT NULL	-	Domain (e.g. Design, Software)
	`website`	VARCHAR(255) NULL	-	Corporate URL
	`bio, logo, socials`	TEXT / VARCHAR NULL	-	Branding assets and links
job_postings	`id`	BIGINT UNSIGNED AUTO_INCREMENT	PRIMARY KEY	Unique vacancy identifier
	`employer_id`	BIGINT UNSIGNED NOT NULL	FOREIGN KEY → employers(id)	Cascade delete employer link
	`title`	VARCHAR(255) NOT NULL	-	Headline gig title
	`job_type`	ENUM('Remote', 'On-site', 'Hybrid')	DEFAULT 'Remote'	Work arrangement category
	`budget`	DECIMAL(10,2) NOT NULL	-	Fixed stipend or project pay
	`deadline, is_active`	DATE, BOOLEAN DEFAULT TRUE	-	Expiry date & listing visibility toggle
job_applications	`id`	BIGINT UNSIGNED AUTO_INCREMENT	PRIMARY KEY	Application submission identifier
	`job_id`	BIGINT UNSIGNED NOT NULL	FOREIGN KEY → job_postings(id)	Cascade delete job reference
	`applicant_id`	BIGINT UNSIGNED NOT NULL	FOREIGN KEY → users(id)	Cascade delete applicant reference
	`cover_letter`	TEXT NOT NULL	-	Candidate pitch message
	`portfolio_url, cv_file, status`	VARCHAR NULL, ENUM	DEFAULT 'Pending'	Candidate links, PDF path, status
bookmarks	`id`	BIGINT UNSIGNED AUTO_INCREMENT	PRIMARY KEY	Watchlist bookmark identifier
	`user_id`	BIGINT UNSIGNED NOT NULL	FOREIGN KEY → users(id)	Cascade delete user reference
	`job_id`	BIGINT UNSIGNED NOT NULL	FOREIGN KEY → job_postings(id)	UNIQUE(user_id, job_id)

2.3 Foreign Keys & Referential Integrity Matrix

> **Exam Requirement Check:** The exam specification requires at least one foreign key. Ascend Gigs implements **5 Foreign Keys** with explicit `ON DELETE CASCADE` guarantees:

Constraint Name	Foreign Key Column	References	On Delete	Referential Action Rationale
`fk_employers_user`	`employers.user_id`	`users(id)`	CASCADE	If a user account is deleted, their organization profile is automatically removed.

Constraint Name	Foreign Key Column	References	On Delete	Referential Action Rationale
fk_jobs_employer	job_postings.employer_id	employers(id)	CASCADE	If a company is purged, all associated gig vacancies are purged cleanly.
fk_apps_job	job_applications.job_id	job_postings(id)	CASCADE	If a gig is deleted, all historical applications tied to it are deleted.
fk_apps_user	job_applications.applicant_id	users(id)	CASCADE	If a student account is closed, their submissions are removed.
fk_bookmarks_pair	bookmarks.(user_id, job_id)	users(id), job_postings(id)	CASCADE	Maintains watchlists; unique composite key prevents duplicate bookmarks.

3. IMPLEMENTATION & MVC ARCHITECTURE [RUBRIC: 50 POINTS]

3.1 Laravel MVC Architectural Breakdown

The system adheres strictly to the Model-View-Controller design pattern:

- **Model Layer (`app/Models/*`)**: Encapsulates database tables into rich Eloquent objects (`User`, `Employer`, `JobPosting`, `JobApplication`, `Bookmark`). Defines relationships, scopes (`scopeFilter`, `scopeActive`), accessors (`getLogoUrlAttribute`, `cvUrl`), and business rules.
- **Controller Layer (`app/Http/Controllers/*`)**: Handles incoming HTTP requests, invokes business logic, enforces authorization gates (e.g. `authorizeOwner()`), coordinates model persistence, and returns views or redirects with flash messages.
- **View Layer (`resources/views/*`)**: Composed of lightweight, semantic Blade templates utilizing component inheritance via `layouts/ascend.blade.php`, styled with responsive CSS and glassmorphic aesthetics.

3.2 Authentication Subsystem

- **Registration (`POST /register`)**: Dual-role account provisioning. Selecting "Employer" dynamically reveals company name and industry inputs in the UI and automatically instantiates the associated `Employer` profile record in a transactional flow.
- **Login (`POST /login`)**: Authenticates credentials using `Auth::attempt()` with Bcrypt password verification and `session()->regenerate()` to block session fixation attacks. Includes quick 1-click demo test credentials.
- **Logout (`POST /logout`)**: Terminates sessions cleanly via `Auth::logout()`, invalidating session storage and regenerating CSRF tokens.

3.3 Detailed Breakdown of the 8 Core Features

#	Feature Name	Route & Method	Controller & Action	Functional Description
1	Job Board Search & Filters	`GET /jobs`	`JobBoardController@index`	Keyword search across title, requirements, description, and company name; filter by job type (Remote, On-site, Hybrid) and budget ranges; 4 sorting algorithms.
2	Job Posting CRUD	`GET|POST|PUT|DELETE /employer/jobs`	`JobPostingController`	Full lifecycle gig management: draft, publish, edit, delete, and 1-click status toggle between active and closed listings.
3	Application Submission	`POST /jobs/{job}/apply`	`JobApplicationController@store`	Student freelancer pitch submission with pitch cover letter, portfolio URL, and uploaded PDF CV resume handled via `Storage::disk('public')`.

#	Feature Name	Route & Method	Controller & Action	Functional Description
4	Employer Applicant Review Dashboard	`GET /employer/applications` `PATCH .../status`	`JobApplicationController@employerReview` `@updateStatus`	Multi-gig candidate review dashboard; filtering by gig and review stage; reviewing pitch text, opening portfolio URLs, downloading PDF CVs, and updating status (Pending, Interviewed, Hired, Rejected).
5	Applicant Status Tracker	`GET /my-applications`	`JobApplicationController@myApplications`	Student freelancer portal tracking all submitted pitches in real-time with color-coded status badges, company details, and stipend metadata.
6	Job Bookmarking / Save List	`POST /jobs/{job}/bookmark` `GET /saved-jobs`	`BookmarkController@toggle` `@index`	1-click toggle save/remove buttons on cards and detail pages; dedicated personal watchlist dashboard for organizing future applications.
7	Application Withdrawal	`DELETE /my-applications/{app}/withdraw`	`JobApplicationController@withdraw`	Permits candidates to withdraw applications before employer review; automatically purges the candidate's uploaded PDF CV resume from disk.
8	Employer Company Profile CRUD	`GET /employer/profile` `PUT /employer/profile`	`EmployerProfileController@edit` `@update`	Company branding portal: updating company bio, sector, website, social links (LinkedIn, Twitter/X, GitHub), and logo image uploads.

3.4 Automated Test Suite Verification

> **Test Coverage:** 12 test cases with 47 automated assertions passing in 1.56s using PHPUnit and Laravel Feature Testing.

```
$ php vendor/phpunit/phpunit/phpunit PHPUnit 11.3.6 by Sebastian Bergmann and contributors. Runtime: PHP 8.3.33
Configuration: C:\Users\Rinaltra\Documents\GitHub\WebProgramming\Midterm\phpunit.xml ..... 12 / 12 (100%) Time:
00:01.564, Memory: 32.00 MB OK (12 tests, 47 assertions)
```

4. ANALYSIS & EVALUATION [RUBRIC: 25 POINTS]

4.1 Architectural Evaluation of Laravel MVC

The Model-View-Controller pattern enforces strict **separation of concerns**. In Ascend Gigs, database schemas and relationships reside solely in Eloquent models, business workflow orchestration is restricted to controllers, and presentation is encapsulated in Blade templates. This decoupling delivers three major advantages:

- **Maintainability**: Modifying database columns or cascade rules requires touching only migrations and models without refactoring UI templates.
- **Testability**: Controllers and models can be unit-tested in isolation via automated HTTP and database test mocks without browser dependency.
- **Parallel Team Development**: Backend engineers can construct API endpoints and seeders simultaneously while frontend designers polish Blade layouts.

4.2 Eloquent ORM vs. Raw SQL & Eager Loading Audit

Laravel's Eloquent ORM translates database records into active-record instances. Compared to raw SQL queries, Eloquent enhances developer productivity and enforces compile-time safety. To prevent the notorious **$N+1$ query defect**, all relationship queries in Ascend Gigs utilize eager loading:

```
// Inefficient Lazy Loading: 1 initial query + N queries for each employer $jobs = JobPosting::all(); // 1 query foreach ($jobs as $job) { echo $job->employer->company_name; // N queries! Total: 1 + N queries } // Optimized Eager Loading (Implemented in Ascend Gigs): $jobs = JobPosting::with('employer')->where('is_active', true)->paginate(9); // Exactly 2 queries executed regardless of the number of jobs displayed!
```

4.3 Database Normalization Audit

Our 5-table relational schema adheres strictly to Third Normal Form (3NF):

- **First Normal Form (1NF)**: Every column contains atomic, scalar values. No repeating groups or nested arrays exist. Every table possesses an unambiguous primary key (`id`).
- **Second Normal Form (2NF)**: All non-key attributes are fully functionally dependent on the primary key. In composite junction tables like `bookmarks`, attributes depend strictly on the whole key pair (`user_id, job_id`).
- **Third Normal Form (3NF)**: No transitive functional dependencies exist. For instance, employer metadata (`company_name`, `website`, `bio`) is stored exclusively in `employers` rather than redundantly duplicated across each `job_postings` record.

4.4 Security Evaluation

Vulnerability Vector	Defense Mechanism in Ascend Gigs	Implementation Detail
Cross-Site Request Forgery (CSRF)	Synchronizer Token Pattern	All state-mutating forms (POST, PUT, PATCH, DELETE) incorporate encrypted `@csrf` tokens verified by Laravel's `VerifyCsrfToken` middleware.
SQL Injection (SQLi)	PDO Prepared Statements	Eloquent ORM executes all queries via parameterized PDO statements, rendering SQL injection vectors impossible even in search filters.
Cross-Site Scripting (XSS)	Automatic HTML Escaping	Blade templating uses `{{ $variable }}` which passes all user-submitted text through `htmlspecialchars(ENT_QUOTES)` before rendering.
Broken Access Control	Ownership Authorization Gates	`JobPostingController@authorizeOwner` verifies that the authenticated user matches `$job->employer_id` before permitting edits, status toggles, or deletions.
Malicious File Upload	MIME Type & Size Restrictions	`StoreJobApplicationRequest` strictly validates `'cv_file' => 'nullable|file|mimes:pdf|max:5120'`, and stores uploads on isolated disks.

4.5 System Limitations & Future Improvements

- **Real-time Notifications**: Currently, candidates check statuses by polling the status tracker. Integrating Laravel Reverb / WebSockets will deliver push notifications on status changes.
- **In-app Candidate Chat**: Integrating direct messaging between employers and shortlisted candidates would eliminate reliance on external email.
- **Stipend Escrow**: Integrating a campus payment gateway (e.g. Midtrans or Stripe) would allow employers to lock stipends into escrow upon hiring.

5. PROJECT CONCLUSION [RUBRIC: 5 POINTS]

The **Ascend Freelance Job Board & Candidate Application Portal** successfully fulfills all specifications outlined in the EF234301 Web Programming Midterm Examination. By leveraging Laravel 11's MVC architecture, robust Eloquent ORM relations, FormRequest validation, and modern Blade templating, the application delivers a seamless experience for campus micro-recruitment.

Key milestones achieved include:

- Complete implementation of 8 core business features exceeding the minimum rubric requirements.
- Secure session-based authentication supporting dual user roles (Freelancers and Employers).
- A normalized, relational schema featuring 5 foreign keys with cascade delete integrity.
- 100% passing automated feature test suite (12 tests, 47 assertions).
- Publication of complete source code to GitHub and deployment for remote accessibility.

6. INDIVIDUAL CONTRIBUTIONS STATEMENT

Team Member Name	Student ID (NRP)	Work Percentage	Core Modules & Deliverables Completed
Rianti Dewi	5025231001	**50%**	• Database architecture, migrations, and Eloquent models. • Authentication system (Register, Login, Role assignment). • Feature 1: Vacancy Search & Multi-criteria Filtering. • Feature 2: Job Posting CRUD & Status Toggle. • Feature 3: Application Submission & PDF CV Upload. • Feature 8: Employer Company Profile CRUD & Logo Storage.
Darminto Utomo	5025231002	**50%**	• Feature 4: Employer Candidate Review Dashboard & Status Workflow. • Feature 5: Freelancer Live Application Status Tracker. • Feature 6: Job Bookmarking & Personal Watchlist. • Feature 7: Candidate Application Withdrawal. • Responsive UI design, Three.js WebGL canvas, and CSS styling. • Automated PHPUnit test suite construction and web deployment.

7. ACADEMIC INTEGRITY PLEDGE

> *"By the name of Allah (God) Almighty, I hereby pledge and declare that I have completed the Midterm Exam independently. I have not engaged in cheating, plagiarism, or received unauthorized assistance in any form. I further declare that any use of AI tools was limited to a supportive role (such as for grammar checking or debugging), and that the final solution is the product of my own intellectual effort. I understand that I will accept all consequences if I am found to have violated this academic integrity pledge."*

Surabaya, 28 October 2026

Signed,

Rianti Dewi (5025231001) **Darminto Utomo** (5025231002)